

Unit 2: Requirements Engineering & System Modeling

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Requirements Engineering: Functional vs non-functional requirements, product, organizational, and external requirements, NFR metrics. The Software Requirements Document: Structure of the SRS, IEEE 830 standard guidelines, users of an SRS. ...

Table of Contents

Functional vs Non-Functional Requirements	p. 2
The Software Requirements Specification (SRS) & IEEE 830	p. 3
Requirements Engineering Processes & Traceability (RTM)	p. 4
Context & Interaction Models: Use Case Diagrams	p. 5
Interaction Models: Sequence Diagrams	p. 6
Structural Models: UML Class Diagrams	p. 7
Behavioral Models: Activity & State Machine Diagrams	p. 8
Model-Driven Engineering (MDE & MDA)	p. 8
One-Page Rapid Revision Sheet	p. 9
Questions Asked in Past CIE & SEE Papers	p. 9
Practice Problems with Hidden Answers	p. 13

Functional vs Non-Functional Requirements

§ 1

A **software requirement** is a condition or capability needed by a user to solve a problem or achieve an objective. Requirements range from high-level abstract statements of service to detailed formal mathematical specifications.

1. Functional Requirements

Statements of services the system should provide, how the system should react to particular inputs, and how the system should behave in particular situations. (In some cases, they also state what the system should *not* do).

Example: "The system shall allow students to search for courses by department, course code, or instructor name."

2. Non-Functional Requirements (NFRs)

Constraints on the services or functions offered by the system. They include timing constraints, constraints on the development process, standards, security policies, and performance targets.

Classification	Sub-Category	Engineering Definition & Concrete Example
Product Requirements (Constrain runtime behavior)	Performance / Speed	The system shall render search query results in less than 0.5 seconds under a load of 1,000 concurrent queries.
	Reliability & Availability	The system shall maintain an uptime availability of 99.9%, with Mean Time Between Failures (MTBF) $\geq$ 720 hours.
	Security & Usability	All stored passwords shall be hashed using bcrypt (cost 12); user interface shall comply with WCAG 2.1 AA accessibility guidelines.
Organizational Requirements (Constrain policies & processes)	Development Standards	All backend services must be written in Java 17 and follow the Google Java Style Guide with 80% test coverage.
	Delivery & Operational	The system must be containerized using Docker and deployable onto Kubernetes clusters.

Classification	Sub-Category	Engineering Definition & Concrete Example
External Requirements (Arise from external factors)	Regulatory / Privacy	Patient medical data must comply with HIPAA regulations and GDPR data erasure guidelines.
	Ethical / Legal	The system shall not collect geographic location data without explicit, opt-in consent from the user.

Quantifying Non-Functional Requirements with Metrics

Requirements engineers must state non-functional constraints quantitatively so that verification can be objectively tested:

Quality Property	Quantitative Measure / Metric
Speed / Latency	Processed transactions/sec, response time to user event (milliseconds), screen refresh rate.
Size / Footprint	Megabytes of RAM consumed, storage disk footprint, binary executable footprint.
Ease of Use	Training time required for user proficiency, number of help frame consultations per task.
Reliability	Mean Time To Failure (MTTF), Probability of Failure on Demand (POFOD), Rate of Failure Occurrence (ROCOF).
Robustness	Time to restart after system crash, percentage of incidents causing data corruption.
Portability	Percentage of target-dependent statements, number of target systems supported.

The Software Requirements Specification (SRS) & IEEE 830

§ 2

The **SRS** is the official statement of what system developers must implement. It serves as a contract between system customers and software developers.

IEEE 830 Section	Standard Contents & Engineering Purpose
1. Introduction	1.1 Purpose, 1.2 Scope of the product, 1.3 Definitions, acronyms, and abbreviations, 1.4 References, 1.5 Overview of document layout.
2. Overall Description	2.1 Product perspective (relation to other systems), 2.2 Product functions summary, 2.3 User characteristics, 2.4 General constraints, 2.5 Assumptions and dependencies.
3. Specific Requirements	External interfaces (UI, hardware, software, communications), Detailed Functional Requirements, Performance constraints, Design constraints, Security, Software system attributes.

Attributes of a High-Quality SRS

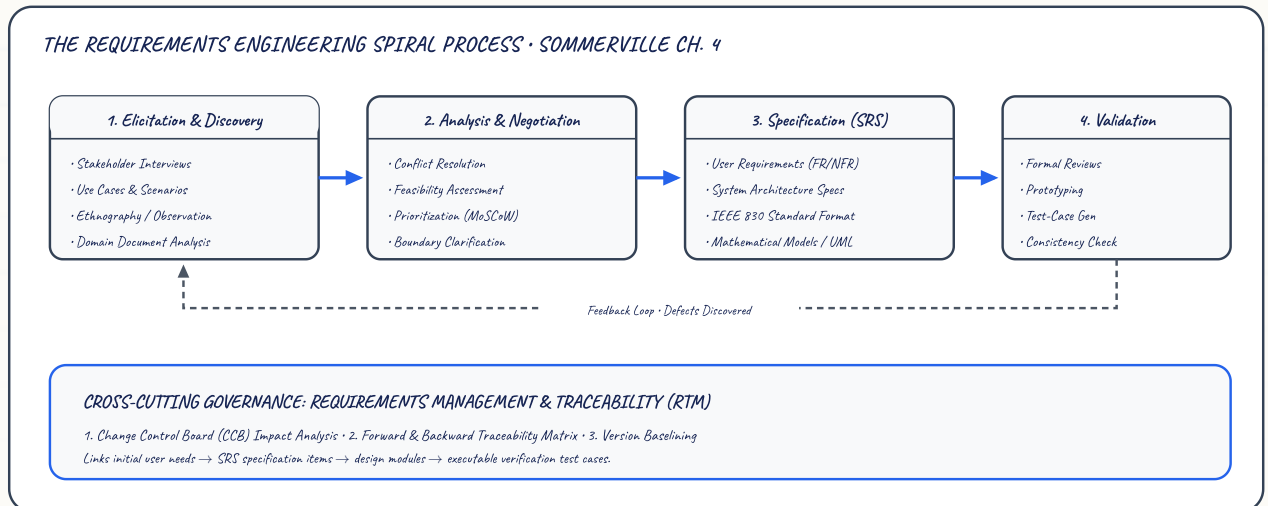
- **Correctness:** Every requirement accurately reflects real stakeholder needs.
- **Unambiguous:** Each requirement has exactly one clear interpretation across all readers.
- **Complete:** All significant requirements (functional, quality, constraints) are fully stated.
- **Consistent:** No requirements conflict with one another.
- **Verifiable:** A cost-effective test or demonstration can prove the software satisfies the requirement.
- **Modifiable:** Structure allows changes to be made easily and consistently.
- **Traceable:** The origin of each requirement is clear, and forward references to design/tests exist.

Requirements Engineering Processes & Traceability (RTM)

§ 3

Requirements engineering is an iterative, spiral process comprising four interdependent activities: Elicitation, Analysis, Specification, and Validation, under continuous Requirements Management.

Figure 2.3: Requirements Engineering Process Flow & RTM Governance (Sommerville Ch. 4)



Requirements Traceability Matrix (RTM)

An **RTM** is a grid that establishes forward and backward traceability between user needs, functional requirements, architectural components, and automated test cases.

Context & Interaction Models: Use Case Diagrams

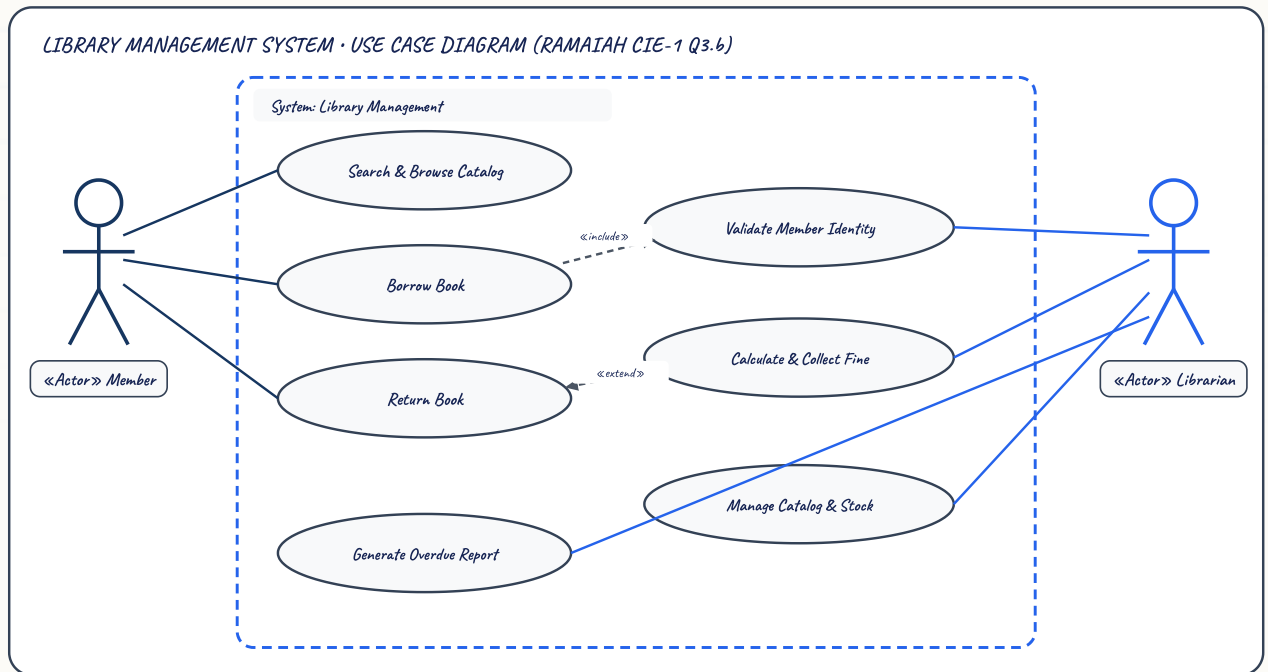
§ 4

System modeling uses graphical notations (principally the Unified Modeling Language - UML) to represent different perspectives of a system: *External context, Interaction, Structural, and Behavioral.*

Use Case Modeling

A **Use Case** describes an interaction between the system and external actors (humans or external hardware/software systems). Actors are represented by stick figures, use cases by horizontal ellipses, and relationships by solid or dashed arrows.

Figure 2.1: Library Management System Use Case Diagram (Ramaiah CIE-1 Oct 2025 Q3.b)



INCLUDE VS EXTEND RELATIONSHIPS IN USE CASES:

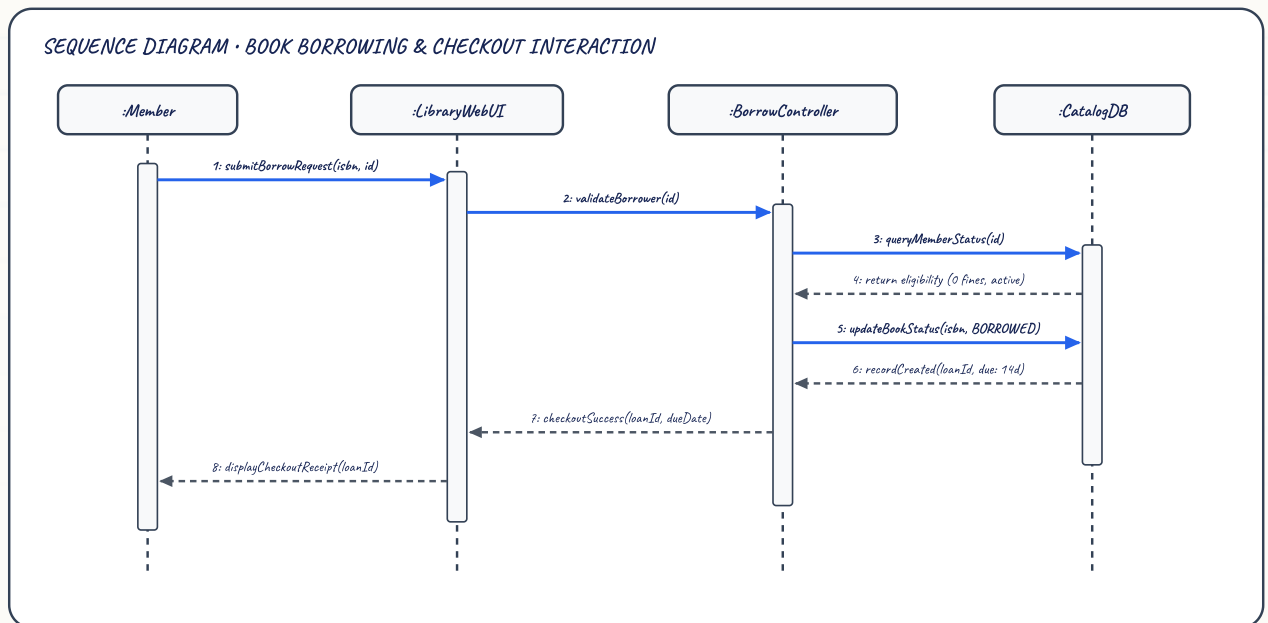
- **<<include>>**: Mandatory behavior. The base use case always executes the included use case as an essential sub-step (e.g., *Borrow Book* <<include>> *Validate Member Identity*).
- **<<extend>>**: Optional / Conditional behavior. The extension use case is executed only under specific circumstances governed by an extension point (e.g., *Return Book* <<extend>> *Calculate & Collect Fine* when loan period > 14 days).

Interaction Models: Sequence Diagrams

§ 5

A **Sequence Diagram** models the dynamic interaction between objects in a use case by depicting the chronological exchange of messages along vertical **lifelines**.

Figure 2.2: Sequence Diagram for Book Borrowing & Checkout Interaction

**KEY SEQUENCE DIAGRAM NOTATIONS:**

- **Lifeline:** Dashed vertical line representing the object's existence over time.
- **Activation Bar:** Vertical rectangle on the lifeline indicating when the object is actively executing an operation.
- **Synchronous Call:** Solid line with filled arrow ($\longrightarrow$) blocking caller until return.
- **Reply / Return:** Dashed line with open arrow ($\dashrightarrow$) conveying computed return values.

Structural Models: UML Class Diagrams**§ 6**

Structural models show the organization and architecture of a system in terms of the components that make up that system and their relationships.

Relationship	UML Notation	Semantics & Example
Association	Solid line (optional navigability arrow)	General structural relationship. <i>Teacher teaches Student.</i>
Aggregation	Solid line with hollow diamond at container	Weak "has-a" relationship. Part can exist without whole. <i>Department has Professors.</i>
Composition	Solid line with filled diamond at container	Strong "has-a" relationship. Part lifetime tied to whole. <i>Building has Rooms.</i>
Generalization	Solid line with hollow triangular arrowhead	"Is-a" inheritance relationship. <i>CheckingAccount is a BankAccount.</i>

Behavioral Models: Activity & State Machine Diagrams

§ 7

Behavioral models capture what happens or what is supposed to happen when a system responds to a stimulus from its environment.

1. Activity Diagrams

Model the procedural flow of control from activity to activity. Swimlanes separate activities according to which subsystem or actor performs them.

2. State Machine Diagrams (Statecharts)

Show the system or object life cycle as a sequence of states triggered by external events. Standard notation: initial state (solid circle), states (rounded rectangles), transitions (arrows with event [guard] / action), final state (bullseye circle).

Model-Driven Engineering (MDE & MDA)

§ 8

Model-Driven Engineering (MDE) is an approach to software development where models, rather than programs, are the principal outputs of the development process.

Layer	Full Name	Role in Model-Driven Architecture (MDA)
CIM	Computation Independent Model	Models domain abstractions without software system concepts (business domain model).
PIM	Platform Independent Model	Models system operation and structure without reference to implementation technology.
PSM	Platform Specific Model	Transforms PIM by adding implementation-specific rules (e.g. Java Spring, PostgreSQL).

One-Page Rapid Revision Sheet

★ SUMMARY

Concept	Core Definition	Exam Key Point
Functional Req	Services and functions the system must execute.	"Shall do X when Y occurs."
Non-Functional Req	Constraints on services (speed, reliability, security).	Must be quantitatively testable (MTBF, response ms).
Availability Formula	$A = \frac{MTBF}{MTBF + MTTR}$	$MTBF = 500h, MTTR = 2h \implies A = 99.60\%$
SRS IEEE 830	Structured standard for requirements documentation.	Sections 1 (Intro), 2 (Description), 3 (Specific Reqs).
Use Case Diagram	Actor-system interaction boundary.	<<include>> mandatory, <<extend>> optional.
Sequence Diagram	Time-ordered message trace across lifelines.	Lifelines, activation bars, sync calls, reply returns.
Aggregation vs Comp	Aggregation: hollow diamond (independent lifecycle).	Composition: filled diamond (cascaded lifecycle).
MDA Hierarchy	CIM → PIM → PSM → Executable Code.	Automated model transformation engines.

Questions Asked in Past CIE & SEE Papers

ARCHIVE

The following authentic questions have been transcribed directly from official department examination papers in notes/se/practice/se-cie-1-and-2.pdf:

Ramaiah Autonomous Examination · CIE-1 (Oct 28, 2025) Q1.b [7 Marks]

"Differentiate between Functional and Non-Functional Requirements. Give two concrete examples of each in the context of an E-Commerce Web Application."

Official Model Solution:

1. Definitions:

- *Functional Requirements (FRs)*: Specify the functional services, business rules, inputs, data processing, and output transformations the software must deliver.
- *Non-Functional Requirements (NFRs)*: Define quality attributes, constraints, environmental conditions, and behavioral boundaries placed on the functional services.

2. E-Commerce Examples:

- *FR-1*: "The system shall allow registered customers to add items to a persistent cart and apply promotional coupon codes during checkout."
- *FR-2*: "The system shall automatically generate and email a PDF invoice within 60 seconds of successful payment gateway confirmation."
- *NFR-1 (Performance)*: "The checkout page shall complete end-to-end processing in under 1.5 seconds under a concurrent traffic load of 5,000 active shoppers."
- *NFR-2 (Security & Compliance)*: "All customer payment card credentials must be tokenized according to PCI-DSS Level 1 compliance; no raw CVV numbers shall be persisted to the database."

<i>Elicitation Challenge</i>	<i>Root Cause / Manifestation</i>	<i>Engineering Mitigation Strategy</i>
Scope Creep & Boundary Confusion	Stakeholders request non-essential features, expanding system boundaries uncontrollably.	Establish a clear Project Charter with a Change Control Board (CCB) and MoSCoW prioritization.
Tacit Knowledge / Articulation Gap	Users know how to do their daily tasks but cannot articulate exact procedural steps.	Deploy Ethnography, on-site observation, and interactive user journey walkthroughs.
Conflicting Stakeholder Needs	Marketing, security, and finance departments place contradictory demands on system behavior.	Conduct joint requirements negotiation sessions with weighted objective matrices.
Evolving / Volatile Requirements	Market conditions and regulatory standards shift during development cycles.	Adopt Agile incremental sprints with short sprint review demos and living product backlogs.

Ramaiah Autonomous Examination · CIE-I (Oct 28, 2025) Q3.b [5 Marks]

"Draw a neat Use Case Diagram for a Library Management System with Member and Librarian actors, including at least three use cases such as Search Books, Borrow Book, Return Book, Manage Catalog, and Fine Calculation."

Official Model Solution:

See **Figure 2.1** above for the verified complete UML Use Case Diagram:

- **Actors:** Member (primary user on left) and Librarian (administrative staff on right).
- **Core Use Cases:** Search & Browse Catalog, Borrow Book, Return Book, Manage Catalog & Stock, Generate Overdue Report.
- **Relationships:**
 - Borrow Book <<include>> Validate Member Identity (mandatory authorization prerequisite).
 - Return Book <<extend>> Calculate & Collect Fine (conditional execution triggered only when the return date exceeds the 14-day checkout limit).

Ramaiah Autonomous Examination · CIE-I (Oct 28, 2025) Q3.c [4 Marks]

"Explain the requirements validation process. Outline three distinct techniques used to validate software requirements."

Official Model Solution:

Requirements Validation is the process of checking that requirements accurately define the system the customer truly desires. Validation prevents costly downstream defects.

1. **Requirements Reviews / Inspections:** A multidisciplinary team of clients, architects, and testers systematically inspects the SRS line-by-line to verify completeness, consistency, realism, and lack of ambiguity.
2. **Prototyping:** Developing throwaway or executable UI prototypes allowing stakeholders to experiment with proposed interfaces and detect missing requirements early.
3. **Test-Case Generation:** If a requirement is realistic and testable, writing an automated acceptance test case for it must be straightforward. Inability to formulate a clear test case indicates the requirement is vague or unachievable.

Practice Problems with Hidden Answers

▽ PRACTICE

PRACTICE 1 · AVAILABILITY CALCULATION

A cloud server has an MTBF of 500 hours and an MTTR of 2 hours. Calculate its availability percentage and total expected annual downtime.

 Practice Problem:

Show Step-by-Step Solution

Solution:

Formula for system availability:

$$A = \frac{\text{MTBF}}{\text{MTBF} + \text{MTTR}} = \frac{500}{500 + 2} = \frac{500}{502} \approx 0.996016 = \mathbf{99.60\%}$$

Annual downtime over a standard 365-day calendar year (8,760 hours):

$$\text{Annual Downtime} = 8760 \times (1 - A) = 8760 \times \left(1 - \frac{500}{502}\right) =$$

Verification: Verified byte-for-byte in
audit/verify/se/verify_se_u2.py.

PRACTICE 2 · NFR CLASSIFICATION

Classify the following requirement according to Sommerville's 3-tier taxonomy: "The system must conform to ISO 9001 quality audit documentation guidelines."

 Practice Problem:

Show Step-by-Step Solution

Solution:

Classification: Organizational Requirement · Development Standards.

Rationale: It governs internal organizational engineering processes, documentation standards, and quality management protocols rather than the runtime computational behavior of the software product (Product Requirement) or external government statutes (External Requirement).

PRACTICE 3 · REQUIREMENTS TRACEABILITY MATRIX (RTM)

In an audit of 6 system requirements, all 6 are successfully mapped to architectural modules and automated test cases. Calculate the Traceability Coverage and describe why backward traceability is vital.

 Practice Problem:

Show Step-by-Step Solution

Solution:

$$\text{Traceability Coverage} = \frac{N_{\text{mapped}}}{N_{\text{total}}} \times 100\% = \frac{6}{6} \times 100\% = 100\%$$

Importance of Backward Traceability:

- Ensures that every line of source code and design module traces back to an authentic, approved user requirement (preventing unauthorized feature bloat or "gold plating").
- When a customer amends or deletes a requirement, backward traceability allows engineers to rapidly isolate and modify only the affected modules and tests without breaking unrelated subsystems.

PRACTICE 4 · USE CASE INCLUDE VS EXTEND CRITERIA

An airline reservation system has a "Book Flight" use case and an "Opt for Travel Insurance" use case. Should the relationship be <<include>> or <<extend>>? Justify your answer.

Practice Problem:

Show Step-by-Step Solution

Solution:

Relationship: <<extend>> (Book Flight ← Opt for Travel Insurance with extension point: "During Payment Review").

Justification:

- Purchasing travel insurance is optional and conditional. A passenger can successfully complete a booking without purchasing insurance.
- <<include>> would mandate that insurance is bought on every single flight booking, which violates domain business logic.

PRACTICE 5 · SEQUENCE DIAGRAM ACTIVATION BAR DURATION

In a sequence diagram, object A sends a synchronous message to object B, which then queries object C. How should the activation bars on A, B, and C be drawn?

Practice Problem:

Show Step-by-Step Solution

Solution:

1. **Object A:** Its activation bar begins when it initiates the workflow and remains active (waiting in blocking state) until it receives the return message from B.
2. **Object B:** Its activation bar begins upon receiving message 1 from A. It stays active throughout its execution, spanning the invocation of C and receipt of C's reply, until it returns control to A.
3. **Object C:** Its activation bar begins when B calls it and terminates immediately once it sends its return data back to B.

PRACTICE 6 · MODEL-DRIVEN ARCHITECTURE (MDA)

Trace the transformation chain in Model-Driven Architecture (MDA) from initial requirements down to executable application code.

Practice Problem:

Show Step-by-Step Solution

Solution:

1. **CIM (Computation Independent Model)**: Domain and business process model capturing user concepts without software system structures.
2. ↓ **[Domain Analysis Transformation]** ↓
3. **PIM (Platform Independent Model)**: Formal UML classes, state machines, and interactions specifying system behavior completely independently of target runtime platform.
4. ↓ **[Platform Mapping Transformation Rules]** ↓
5. **PSM (Platform Specific Model)**: Detailed models customized for target hardware/middleware (e.g. Java Spring JPA entities, RESTful endpoints).
6. ↓ **[Automated Code Synthesis]** ↓
7. **Executable Application Code**: Generated source files and database DDL scripts ready for deployment.